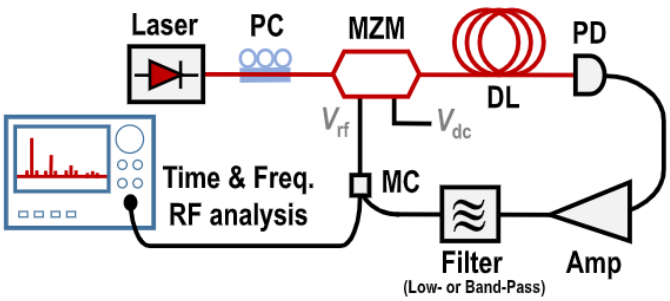
MPL Work Report

Cyrus Young

1) Introduction to OEOs:

1.1) Theory:

Imagine we have a Optoelectronic Oscillator (OEO) that is an Ikeda-like system shown below:



Imagine we want to measure the intensity of the optical beam at the point below the MZM (Mach-Zehnder Modulator) and the MC (Microwave coupler). Let's call this point A. Because we are in an oscillator circuit, the intensity of the laser at that point will likely change over time. It would be nice to know how the intensity at that point would change over time. We do this by representing each block/component as a mathematical function.

At point A, we have a signal with intensity $x(t)$. It goes through a MZM which applies a nonlinear function to it, so now after the MZM, we have $f_{NL}(x(t))$. Next, it goes through a delay loop and thus undergoes a time delay so our original signal is now $f_{NL}(x(t - \tau))$. It then goes through a photodetector (which is a measurement device and doesn't affect the signal) and then an amplifier and thus the signal after the amplifier is $\beta f_{NL}(x(t - \tau))$. Lastly, the signal goes through a filter (called h), and now we are back at our starting point, point A (it should be noted that the Microwave Coupler does not have an effect on the device). Therefore, we can represent our system via the following equations:

$$x(t) = h * \beta f_{NL}(x(t - \tau))$$

$$h^{-1} * x(t) = \beta f_{NL}(x(t - \tau))$$

If we set $h^{-1} = \hat{H}$, we get the following equation:

$$\hat{H}[x(t)] = \beta f_{NL}(x(t - \tau))$$

And in many cases, $\hat{H}[x(t)]$ is a differential equation with $x(t)$. This is important as it allows us to give us a mathematical model of how the output (usually the voltage or intensity of the signal) at a given point varies over time.

1.2) Option 1: Using Simulink to simulate OEO:

The first option is to use Simulink to simulate the behaviour of the OEO as a function of time. This method, as it turns out, is in many ways inferior compared to the next method (labeled Option 2) due to its lack of transparency and lack of versatility. It lacks transparency as the functions are hidden behind blocks - making it harder to detect errors - and lacks versatility as there is not a block for every component. In any case, this method is mentioned to show an initial approach.

1.3) Option 2: MATLAB Code to Simulate OEO:

As opposed to using a Simulink simulation to model the system, it would be preferable to represent the system entirely via code. This will allow our representation to not only be more versatile (compared to the simulation which could only simulate a limited number of components) since we can now write code for each component, but also more transparent (as we are now writing the functions directly rather than using blocks). Below is the code to simulate an OEO as shown in section 1.1:

```
clear;  rng default          % deterministic noise seed

%% Core Parameters:

fc   = 10e9;                 % RF freq [Hz]
tauD = 20e-6;                % fibre delay 20 µs
Qrf  = 3000;                 % RF band-pass Q
```

```
Glin = 35;                  % small-signal loop gain [dB]
Vpi  = 4;                   % MZM half-wave voltage [V]
phiBias = pi/2;             % quadrature bias for max linearity
Fs = 2e9;   dt = 1/Fs;      % envelope-level sample rate / step
Nt = round(1e-3/dt);        % simulate 1 ms  (>50× τ_D )
Ndelay = round(tauD/dt);    % integer delay in samples
assert(Ndelay>10,'Increase Fs or shorten τ_D');

%% Blocks:

% MZM intensity transfer, bias φb
mzm  = @(v) cos(pi*v/(2*Vpi) + phiBias).^2;

% Soft-limiting RF amplifier (unit saturation, Glin dB small-signal)
A0 = 10^(Glin/20);
amp = @(x) (A0*x)./max(A0,abs(x));     % gentle bend into 1 V max

% Base-band equivalent of RF band-pass (2-pole Butterworth LPF)
BW = fc/Qrf;  f3 = BW/2;  wn = 2*pi*f3;                % rad/s cutoff
[b,a] = butter(2,wn/(pi*Fs));                          % digital LPF

%% Run ---------------------------------------------------------------

bufOpt = zeros(Ndelay,1);               % optical-delay circular buffer
y      = complex(zeros(Nt,1));          % RF output record
noise  = (randn(Nt,1)+1i*randn(Nt,1))*1e-6;

% IIR filter state for per-sample processing
zi = zeros(max(numel(a),numel(b))-1,1);
v_prev = 0;                             % previous RF sample feeding the MZM

for n = 1:Nt
   % MZM (intensity transfer) ------------------------------------
   v_mzm = mzm(real(v_prev));          % intensity ∝ cos²(⋯)

   % Fibre delay -------------------------------------------------
   k       = mod(n-1,Ndelay) + 1;      % circular-buffer index
   v_del   = bufOpt(k);                % intensity after τ_D
   bufOpt(k) = v_mzm;                  % write current intensity

   % Photo-detector (linear) -------------------------------------
   v_pd = v_del;                       % voltage ∝ delayed intensity

   % RF amplifier ------------------------------------------------
   v_amp = amp(v_pd);                  % soft-limiting gain

   % Narrowband RF filter ----------------------------------------
   [v_filt, zi] = filter(b,a,v_amp,zi);  % 2-pole Butterworth

   % Close the loop ----------------------------------------------
   y(n)   = v_filt + noise(n);         % record & seed with noise
   v_prev = y(n);                      % feeds next-step MZM
end
```

```
%% Visualise ------------------------------------------------------------

t = (0:Nt-1)*dt;
Y = fftshift(fft(y));
f = Fs*(-Nt/2:Nt/2-1)/Nt;
figure;
subplot(2,1,1), plot(t*1e3, real(y));
xlabel('Time (ms)'), ylabel('v_{RF} (a.u.)'), title('OEO start-up & steady-state');
subplot(2,1,2), plot(f/1e6, 20*log10(abs(Y)/max(abs(Y))));
xlim([fc/1e6-100 fc/1e6+100]), grid on
xlabel('Frequency (MHz)'), ylabel('Magnitude (dB)'), title('Spectrum near 10 GHz');
```

We will now delve into a detailed explanation of the actual functioning of the code. The first step is to define the core parameters of our system. fc represents our oscillation frequency (i.e. the carrier frequency of which the system is defined). tauD represents the loop delay. Qrf represents the Quality factor. Glin represents the small-signal loop gain - we use a small signal as this describes the gain before saturation. Vpi is the Mach-Zehnder Modulators half-wave voltage - the voltage swing needed to go from maximum to minimum optical output.

```
fc    = 10e9;                 % RF freq [Hz]
tauD = 20e-6;                 % fibre delay 20 µs
Qrf   = 3000;                 % RF band-pass Q
Glin = 35;                    % small-signal loop gain [dB]
Vpi   = 4;                    % MZM half-wave voltage [V]
phiBias = pi/2;               % quadrature bias for max linearity
Fs = 2e9;    dt = 1/Fs;       % envelope-level sample rate / step
Nt = round(1e-3/dt);          % simulate 1 ms  (>50× τ_D )
Ndelay = round(tauD/dt);      % integer delay in samples
```

The next step is to make sure that τ_d , which represents the physical fibre time delay, is sufficiently represented in the program in order for the delay to be non-negligible. Our next operation is performing the corresponding MZM (Mach-Zehnder-Modulator) function on it. Here, the MZM applies a nonlinear effect on the signal, *v*, by applying a cos^2 plus phase operation on it.

```
% MZM intensity transfer, bias φb
mzm  = @(v) cos(pi*v/(2*Vpi) + phiBias).^2;
```

We then need to make sure the signal is amplified, and in a realistic fashion (i.e. we want it to eventually converge to some value, *A0*, as our signal becomes larger (thus mimicking the

behavior of actual amplifiers). We do this by giving the following function, *amp(x)*, where *x* is the signal:

```
% Soft-limiting RF amplifier (unit saturation, Glin dB small-signal)
A0 = 10^(Glin/20);
amp = @(x) (A0*x)./max(A0,abs(x));     % gentle bend into 1 V max
```

We then implement a band-pass filter like so. What the following code below implements is bandpass filter with a frequency band ranging from $0 \leq f \leq f_c / 2Q_{rf}$.

```
% Base-band equivalent of RF band-pass (2-pole Butterworth LPF)
BW = fc/Qrf;  f3 = BW/2;  wn = 2*pi*f3;                % rad/s cutoff
[b,a] = butter(2,wn/(pi*Fs));                          % digital LPF
```

The next step is to create the optical-delay circuit buffer.

```
%% Run ---------------------------------------------------------------------

bufOpt = zeros(Ndelay,1);               % optical-delay circular buffer
y      = complex(zeros(Nt,1));          % RF output record
noise  = (randn(Nt,1)+1i*randn(Nt,1))*1e-6;
```

Relevant Papers:

1) *Optoelectronic oscillators with time-delayed feedback*

2) *Tutorial: Photonic neural networks in delay systems*